



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7
по дисциплине «Программирование на языке Джава»

Выполнил студент группы **ИКБО-50-25**

Багинян А.В.

Принял
Старший преподаватель

Ермаков С.Р.

Москва 2026

1. Задачи к выполнению

1.1. Часть 1. Структурные паттерны проектирования

Структурные паттерны (Structural Patterns) — это группа паттернов проектирования, которая фокусируется на способах организации классов и объектов в более крупные структуры. Они обеспечивают гибкость и удобство взаимодействия между компонентами системы, способствуя уменьшению связности и упрощению поддержки кода. К структурным паттернам относятся Adapter, Bridge, Composite, Decorator, Facade, Flyweight и Proxy.

1.1.1. Практическое задание — вариант №4 (Decorator)

Создать систему для расширения функционала базового текстового редактора. Например, добавьте декораторы для форматирования текста (жирный, курсив и подчёркнутый текст).

Требования:

- Реализовать интерфейс `Text` с методом `display()`.
- Создать базовый класс для простого текста и декораторы для добавления форматирования.
- Декораторы должны быть применимы в любом порядке.

Паттерн Decorator динамически добавляет объекту новые обязанности, не изменяя его код: абстрактный класс `TextDecorator` сам реализует `Text` и оборачивает другой объект `Text` (декорируемый или уже декорированный ранее), делегируя ему вызов и добавляя собственное форматирование поверх результата. Поскольку каждый конкретный декоратор (`BoldDecorator`, `ItalicDecorator`, `UnderlineDecorator`) одновременно является `Text` и принимает `Text` в конструкторе, декораторы можно оборачивать друг в друга в любом порядке.

1.1.1.1. Решение

```
1 // patterns/decorator/Text.java
2 package ru.haykor.mirea.patterns.decorator;
3
4 public interface Text {
5     String display();
6 }
```

```
1 // patterns/decorator/PlainText.java
2 package ru.haykor.mirea.patterns.decorator;
```

```

3
4 public class PlainText implements Text {
5     private final String content;
6
7     public PlainText(String content) {
8         this.content = content;
9     }
10
11     @Override
12     public String display() {
13         return content;
14     }
15 }

```

```

1 // patterns/decorator/TextDecorator.java
2 package ru.haykor.mirea.patterns.decorator;
3
4 public abstract class TextDecorator implements Text {
5     protected final Text decoratedText;
6
7     protected TextDecorator(Text decoratedText) {
8         this.decoratedText = decoratedText;
9     }
10 }

```

```

1 // patterns/decorator/BoldDecorator.java
2 package ru.haykor.mirea.patterns.decorator;
3
4 public class BoldDecorator extends TextDecorator {
5     public BoldDecorator(Text decoratedText) {
6         super(decoratedText);
7     }
8
9     @Override
10    public String display() {
11        return "<b>" + decoratedText.display() + "</b>";
12    }
13 }

```

```

1 // patterns/decorator/ItalicDecorator.java
2 package ru.haykor.mirea.patterns.decorator;
3

```

```

4 public class ItalicDecorator extends TextDecorator {
5     public ItalicDecorator(Text decoratedText) {
6         super(decoratedText);
7     }
8
9     @Override
10    public String display() {
11        return "<i>" + decoratedText.display() + "</i>";
12    }
13 }

```

```

1 // patterns/decorator/UnderlineDecorator.java
2 package ru.haykor.mirea.patterns.decorator;
3
4 public class UnderlineDecorator extends TextDecorator {
5     public UnderlineDecorator(Text decoratedText) {
6         super(decoratedText);
7     }
8
9     @Override
10    public String display() {
11        return "<u>" + decoratedText.display() + "</u>";
12    }
13 }

```

```

1 // app/TestTextDecorator.java
2 package ru.haykor.mirea.app;
3
4 import ru.haykor.mirea.patterns.decorator.BoldDecorator;
5 import ru.haykor.mirea.patterns.decorator.ItalicDecorator;
6 import ru.haykor.mirea.patterns.decorator.PlainText;
7 import ru.haykor.mirea.patterns.decorator.Text;
8 import ru.haykor.mirea.patterns.decorator.UnderlineDecorator;
9
10 public class TestTextDecorator {
11     public static void main(String[] args) {
12         Text plain = new PlainText("Привет, мир!");
13         System.out.println(plain.display());
14
15         Text boldThenItalic = new ItalicDecorator(new
BoldDecorator(plain));
16         System.out.println(boldThenItalic.display());

```

```

17
18         Text italicThenBold = new BoldDecorator(new
ItalicDecorator(plain));
19         System.out.println(italicThenBold.display());
20
21         Text allThree = new UnderlineDecorator(new
ItalicDecorator(new BoldDecorator(plain)));
22         System.out.println(allThree.display());
23     }
24 }

```

Вывод программы показывает, что декораторы `BoldDecorator` и `ItalicDecorator` дают разный, но корректный результат в зависимости от порядка оборачивания (`<i><b>...</b></i>` против `<b><i>...</i></b>`), а любое их количество можно комбинировать в цепочку:

```

1 Привет, мир!
2 <i><b>Привет, мир!</b></i>
3 <b><i>Привет, мир!</i></b>
4 <u><i><b>Привет, мир!</b></i></u>

```

1.2. Часть 2. Поведенческие паттерны проектирования

Поведенческие паттерны (Behavioral Patterns) описывают способы взаимодействия объектов и классов друг с другом, фокусируясь на алгоритмах и распределении обязанностей. Они помогают обеспечить эффективное и гибкое взаимодействие между компонентами системы, упрощая коммуникацию и управление поведением объектов. К поведенческим паттернам относятся Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method и Visitor.

1.2.1. Практическое задание — вариант №4 (Mediator)

Реализовать систему чата, где несколько пользователей могут обмениваться сообщениями через посредника.

Требования:

- Создать интерфейс `Mediator` для координации сообщений.
- Реализовать классы `User`, которые будут отправлять и получать сообщения через посредника.
- Продемонстрировать обмен сообщениями между пользователями.

Паттерн Mediator обеспечивает централизованное взаимодействие между объектами, уменьшая их связанность: классы `User` не хранят ссылки друг на

друга и ничего не знают о том, сколько в чате других участников, — они знают только об объекте `ChatMediator`, которому передают исходящие сообщения и от которого получают входящие через обратный вызов `receive()`.

1.2.1.1. Решение

```
1 // patterns/mediator/ChatMediator.java
2 package ru.haykor.mirea.patterns.mediator;
3
4 public interface ChatMediator {
5     void addUser(User user);
6
7     void sendMessage(String message, User sender);
8 }
```

```
1 // patterns/mediator/User.java
2 package ru.haykor.mirea.patterns.mediator;
3
4 public class User {
5     private final String name;
6     private final ChatMediator mediator;
7
8     public User(String name, ChatMediator mediator) {
9         this.name = name;
10        this.mediator = mediator;
11        mediator.addUser(this);
12    }
13
14    public String getName() {
15        return name;
16    }
17
18    public void send(String message) {
19        System.out.println(name + " отправляет: " + message);
20        mediator.sendMessage(message, this);
21    }
22
23    public void receive(String message, String from) {
24        System.out.println(name + " получил от " + from + ": "
25        + message);
26    }
27 }
```

```

1 // patterns/mediator/ChatRoom.java
2 package ru.haykor.mirea.patterns.mediator;
3
4 import java.util.ArrayList;
5 import java.util.List;
6
7 public class ChatRoom implements ChatMediator {
8     private final List<User> users = new ArrayList<>();
9
10    @Override
11    public void addUser(User user) {
12        users.add(user);
13    }
14
15    @Override
16    public void sendMessage(String message, User sender) {
17        for (User user : users) {
18            if (user != sender) {
19                user.receive(message, sender.getName());
20            }
21        }
22    }
23 }

```

```

1 // app/TestChatMediator.java
2 package ru.haykor.mirea.app;
3
4 import ru.haykor.mirea.patterns.mediator.ChatMediator;
5 import ru.haykor.mirea.patterns.mediator.ChatRoom;
6 import ru.haykor.mirea.patterns.mediator.User;
7
8 public class TestChatMediator {
9     public static void main(String[] args) {
10        ChatMediator chatRoom = new ChatRoom();
11
12        User alice = new User("Алиса", chatRoom);
13        User bob = new User("Боб", chatRoom);
14        User carol = new User("Кэрол", chatRoom);
15
16        alice.send("Привет всем!");
17        bob.send("Привет, Алиса!");
18    }
19 }

```

Вывод программы показывает, что каждое сообщение, отправленное через `send()`, посредник `ChatRoom` рассылает всем участникам чата, кроме самого отправителя:

- 1 Алиса отправляет: Привет всем!
- 2 Боб получил от Алиса: Привет всем!
- 3 Кэрл получил от Алиса: Привет всем!
- 4 Боб отправляет: Привет, Алиса!
- 5 Алиса получил от Боб: Привет, Алиса!
- 6 Кэрл получил от Боб: Привет, Алиса!